

Case Analysis of Binary Search Algorithm

CSA0606-Design and Analysis of Algorithm
Assessment Tool – 3
Technical Presentation

Name : V.Yugandhar
Reg id :192472353
Dept : CSE, AI

Introduction

Searching is one of the most important operations in computer science. It is used to find a specific element from a collection of data. Searching algorithms are widely used in applications such as libraries, banking systems, online shopping, student databases, hospital records, and employee management systems.

Definition:

- Binary Search is an efficient searching algorithm that works only on sorted data.
- Instead of checking every element one by one, it compares the search key with the middle element and repeatedly divides the search space into two halves.
- This significantly reduces the number of comparisons and makes searching much faster than Linear Search.

Problem Statement

Analyze the performance of the Binary Search algorithm under best, average, and worst-case scenarios, identify the conditions for each case, represent the time complexity using Big O, Big Ω , and Big Θ , and determine which case is most relevant in real-world applications.

Objectives:

- Understanding the working principle of the Binary Search algorithm.
- Analyzing the best, average, and worst-case scenarios of Binary Search.
- Identifying the conditions for each case.
- Apply asymptotic notation (Big O, Big Ω , and Big Θ) to analyze time complexity.
- Compare the performance of Binary Search under different cases using tables or diagrams.
- Determine which case is most relevant in real-world applications.:

Algorithm

Step 1: Start.

Step 2: Enter the sorted array and the search key.

Step 3: Initialize:

$low = 0;$

$high = n - 1;$

Step 4: Repeat while:

$low \leq high$

Step 5: Find the middle element:

$mid = (low + high) / 2;$

Step 6: Compare the key with arr[mid]:

If arr[mid] == key → Element Found.

If key < arr[mid] → high = mid - 1;

Otherwise → low = mid + 1;

Step 7: If low > high, display "Element Not Found".

Step 8: Stop.

Declarations and Conditions :

➤ Declarations of binary search: int arr[100];
int low, high, mid, key,n;

➤ Conditions of binary search :

- The array must be sorted (ascending or descending order).

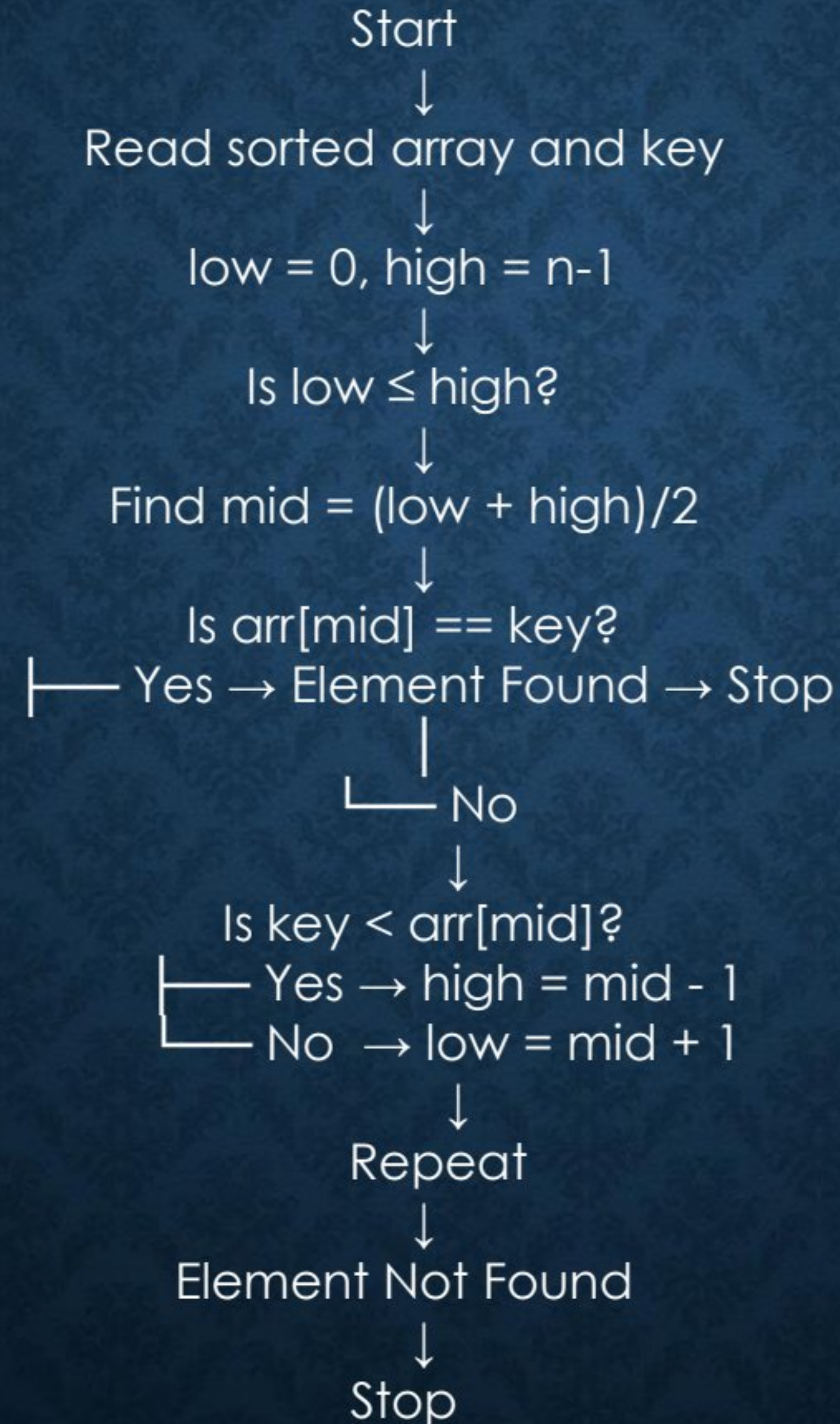
Initialize:

- low = 0
- high = n - 1

Continue searching while:

- low <= high

Architecture Diagram



Case Analysis

1. Best Case Analysis

Condition

- The required element is exactly at the **middle** of the sorted array.
- Element found in **one comparison**.

Asymptotic Notation

$\Omega(1)$ (Best Case)

2. Average Case Analysis

Condition

- The required element is present but not at the middle.
- The algorithm divides the array repeatedly until the element is found.

Asymptotic Notation

$\Theta(\log n)$

3.Worst Case Analysis

Condition

- Element is not present, or
- It is found only after the maximum number of comparisons.

Asymptotic Notation

$O(\log n)$

Comparison Table

Case	Condition	Comparisons	Complexity
Best	Middle element	1	$\Omega(1)$
Average	Element anywhere	$\log n$	$\Theta(\log n)$
Worst	Not found / Last comparison	$\log n$	$O(\log n)$

Relationship with Asymptotic Notation

Asymptotic Notation	Binary Search Case	Meaning
Big Omega (Ω)	Best Case	Minimum time taken = $\Omega(1)$
Big Theta (Θ)	Average Case	Average time taken = $\Theta(\log n)$
Big O (O)	Worst Case	Maximum time taken = $O(\log n)$

- The relation between Asymptotic Notation and Binary Search is that asymptotic notation is used to measure and express the efficiency (time complexity) of the Binary Search algorithm.
- Asymptotic Notation is a mathematical way to describe how the running time of Binary Search grows as the input size (n) increases.

Code:

```
#include <stdio.h>

int main()
{
    int arr[100], n, key;
    int low, high, mid, i;

    // Read number of elements
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Read sorted array
    printf("Enter the sorted array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Read search key
    printf("Enter the element to search: ");
    scanf("%d", &key);
```



```
printf("Enter the element to search: ");  
scanf("%d", &key);  
  
// Initialize variables  
low = 0;  
high = n - 1;  
  
// Binary Search  
while(low <= high)  
{  
    mid = (low + high) / 2;  
  
    if(arr[mid] == key)  
    {  
        printf("Element found at index %d", mid);  
        return 0;  
    }  
    else if(key < arr[mid])  
    {  
        high = mid - 1;  
    }  
}
```



```

else
{
    low = mid + 1;
}
}

printf("Element not found");

return 0;
}

```

```

#include <stdio.h>

int main()
{
    int arr[100], n, key;
    int low, high, mid, i;

    // Read number of elements
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Read sorted array
    printf("Enter the sorted array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Read search key
    printf("Enter the element to search: ");
    scanf("%d", &key);

    // Initialize variables
    low = 0;
    high = n - 1;

```

```

// Binary Search
while(low <= high)
{
    mid = (low + high) / 2;

    if(arr[mid] == key)
    {
        printf("Element found at index %d", mid);
        return 0;
    }
    else if(key < arr[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}

printf("Element not found");

return 0;

```

Output

- Used C programming language.
- Programiz C Online Compiler is used to compile code.
- Finally, Code executed successfully and expected elements are found correctly.

Output

```

Enter the number of elements: 6
Enter the sorted array elements:
10 20 30 40 50 60
Enter the element to search: 30
Element found at index 2

=== Code Execution Successful ===

```


Which Case is Most Relevant in Practice?

The **average case** is the most relevant in real applications because:

- Usually, we do not know where the element is located.
- Sometimes the element is present, and sometimes it is absent.
- Most searches require several comparisons before finding the element.

Therefore, the practical performance of Binary Search is generally $\Theta(\log n)$, making it much faster than Linear Search for large sorted datasets.

Conclusion

- Binary Search is a fast and efficient searching algorithm for sorted arrays.
- It reduces the search space by half in each step, making searching quicker than Linear Search.
- The time complexities are:
 - Best Case: $\Omega(1)$
 - Average Case: $\Theta(\log n)$
 - Worst Case: $O(\log n)$
- The average case is the most relevant in practice because the position of the element is usually unknown.
- Binary Search is widely used in applications such as databases, libraries, banking systems, and search engines due to its high efficiency.

References

1. GeeksforGeeks – Binary Search

<https://www.geeksforgeeks.org/binary-search/>

2. W3Schools – Binary Search Algorithm

https://www.w3schools.com/dsa/dsa_algo_binarysearch.php

3. Programiz – Binary Search

<https://www.programiz.com/dsa/binary-search>

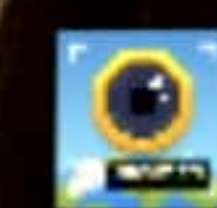
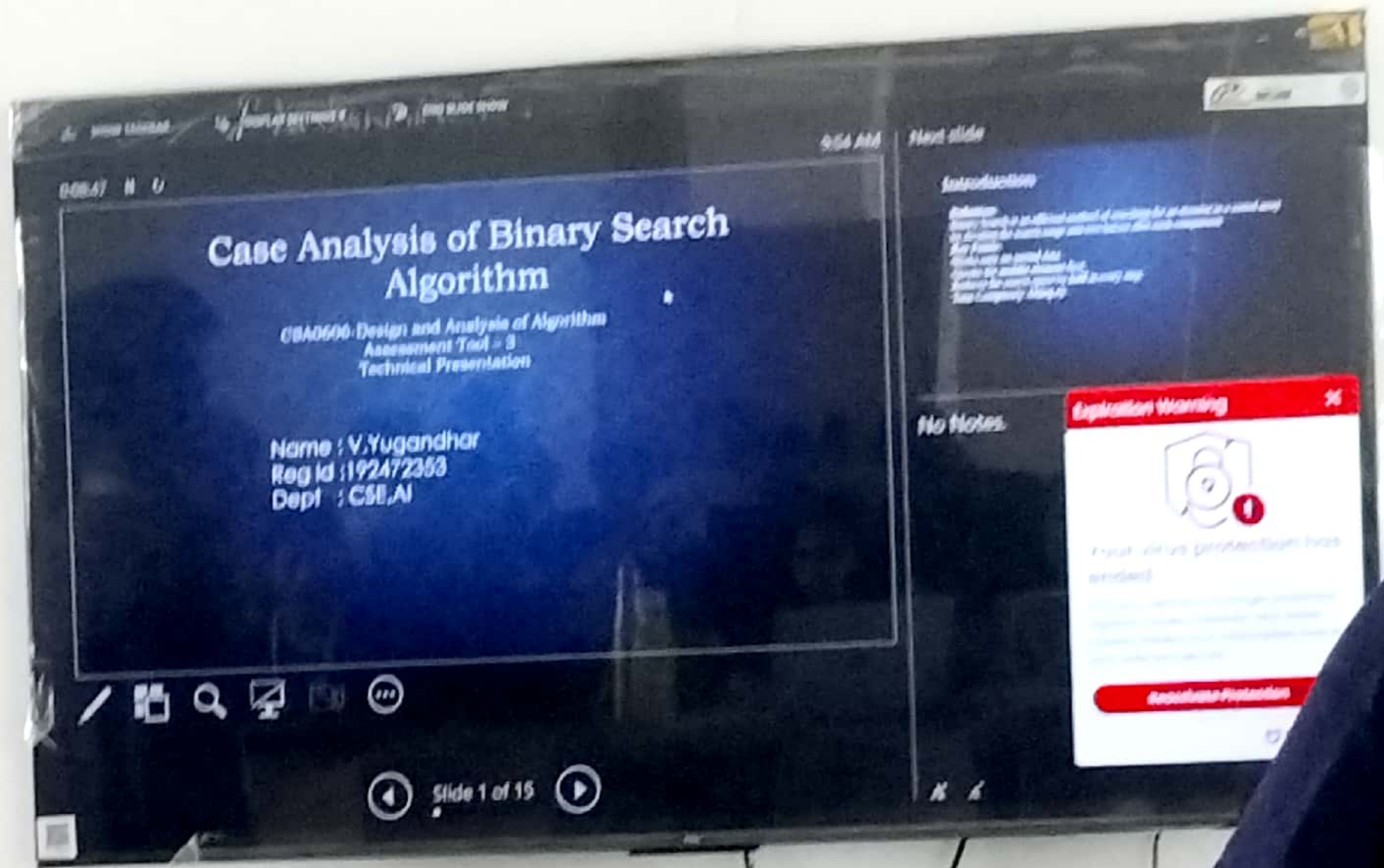
4. TutorialsPoint – Binary Search Algorithm

https://www.tutorialspoint.com/data_structures_algorithms/binary_search_algorithm.htm

5. Javatpoint – Binary Search

<https://www.javatpoint.com/binary-search>

Thank You



GPS Map Camera

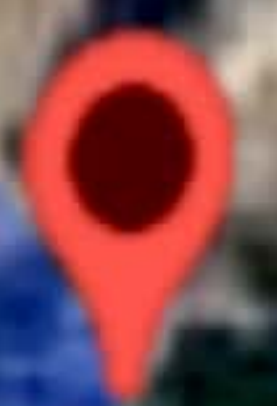
Mevalurkuppam, Tamil Nadu, India



22h8+5h2, Mevalurkuppam, Tamil Nadu 602105, India

Lat 13.027521° Long 80.016735°

Wednesday, 22/07/2026 09:54 AM GMT +05:30



Google